Scaling Storage Capacity with Sharded Databases

1. Why Storage Scaling Becomes Necessary

As applications grow, database size may exceed limits of a single server.

Common limits:

```
None
Disk capacity

IOPS limits

Memory limits

CPU bottlenecks

Backup/recovery time

Replication lag

Cost of vertical scaling
```

At this stage, one machine is no longer enough.

2. What is Sharding?

Sharding = splitting large data into smaller partitions and distributing them across multiple servers.

```
None
Shard 1 → Server A

Shard 2 → Server B

Shard 3 → Server C
```

Each shard stores only part of total data.

This gives near-unlimited horizontal scaling by adding more nodes.

3. Why Use Sharded Storage?

Benefits:

```
None
Scale storage capacity

Scale read/write throughput

Reduce single node bottleneck

Improve parallel processing

Lower cost than giant server

Better fault isolation
```

4. Real Systems That Use Sharding

- Apache Cassandra
- MongoDB
- Apache HBase
- HDFS
- Amazon DynamoDB

5. Example

Suppose users table has 2 billion rows.

Instead of one server:

```
None
users table → 1 huge DB
```

Use:

```
None
users_0_to_9      → DB1

users_a_to_f      → DB2

users_g_to_m      → DB3

users_n_to_z      → DB4
```

6. Common Sharding Strategies

A. Range-Based Sharding

```
None
user_id 1-1M → shard1

1M-2M → shard2
```

Good for range queries.

Risk:

```
None
Hotspot if recent IDs concentrated
```

B. Hash-Based Sharding

```
None
hash(user_id) % N
```

Even distribution.

Harder range scans.

C. Geographic Sharding

```
None
India users → Mumbai cluster

EU users → Frankfurt cluster

US users → Virginia cluster
```

Low latency.

D. Tenant-Based Sharding

```
None
Enterprise customer A → shard1

Customer B → shard2
```

Good for SaaS systems.

7. When Single DB Fills Up

Options:

A. Delete Old Data

If retention not required.

B. Archive Old Data

Move cold data to cheaper storage.

C. Shard Database

Best for long-term growth.

8. Hot + Cold Storage Pattern

Very common architecture:

```
None
Recent active data → SQL / Redis

Historical data → HDFS / S3 / Cassandra
```

Use fast DB for user requests, cheap scalable storage for history.

9. Why Use Redis for Serving Traffic

Redis is useful for:

```
Low latency reads

Sessions

Leaderboards

Counters

Cache layer
```

Use Redis for hot data, not full historical archive.

10. Frontend Storage Option

Can store limited data on user devices:

```
Browser cookies

localStorage

mobile device cache

offline storage
```

Use cases:

```
None
Theme preferences

Recently viewed items

Session tokens

Temporary cart state
```

Limitations:

```
None
User can clear data

Security risk

Limited size

Not trusted source of truth
```

11. Problems in Sharded Databases

A. Rebalancing

When adding new node:

```
None
Move some data to new shard
```

B. Cross-Shard Joins

Harder than single DB joins.

C. Global Transactions

Distributed ACID is complex.

D. Hot Keys

One shard gets too much traffic.

E. Operational Complexity

Backups, monitoring, migrations harder.

12. How to Handle Growth Properly

```
None
1. Keep hot data separate
2. Use retention policy
3. Archive cold data
4. Add shards gradually
5. Use observability
6. Plan rebalancing early
```

13. Interview HLD Best Answer

If asked “database too large, what do you do?”

Say:

```
None
Use sharding for horizontal scale

Use Redis for hot reads

Move old analytics data to HDFS/S3

Use retention + archival policy
```

Strong answer.

14. Example Architecture

```
Users API
   ↓
MySQL (recent transactions)
   ↓
Kafka
   ↓
Cassandra / HDFS / S3 (historical data)
   ↓
Analytics / BI / ML
```

15. Advantages

```
Infinite-like horizontal growth

Lower cost per TB

Better throughput

Better analytics separation
```

16. Disadvantages

None

Complexity

Shard management

Cross-shard queries

Data movement cost

Consistency harder

17. One-Line Summary

When a single database cannot hold growing data, use sharded storage systems like Cassandra or HDFS, keep hot data in fast databases like Redis, and archive cold data separately for scalable long-term growth.